

1. Introduction

- ◆ Practical Extraction and Report Language (PERL)
- ◆ History
 - Creator: Larry Wall
 - Version: Version 4 and new Version 5 (Object-Oriented)
- ◆ Purpose:
 - For Unix users.
 - Shell scripts may not be portable.
 - C or other language may be too complicated.

Hello World

```
#!/usr/bin/perl  
print "Hello, World!\n";  
print "What's your name? ";  
$name = <STDIN>;  
chop($name);  
print "Welcome $name.\n";
```

Running Perl on Unix

```
%hello.pl
```

```
Hello, World!
```

```
What's your name? Stephen Huang
```

```
Welcome Stephen Huang.
```

Simple Rules to start with

- ◆ Perl is an interpreter. No object code is produced. The “script” can be run directly.
- ◆ Perl is a free-format language. White space can be used freely. Programs are normally indented.
- ◆ There is no “main” routine. There may be functions called “sub”.

Simple Rules to start with

- ◆ Comments start with a # and continue to the end of a line. Anything after the # on the line will be ignored.
- ◆ There are no multiline comments (e. g., C).
- ◆ Case-sensitive.
- ◆ A Perl program consists of several statements.
- ◆ Each statement is terminated by a semicolon (;).

Simple Rules to start with

- ◆ General format of a statement
 - operator (arg1, arg2, ..., argN).
- ◆ Most of the time, we can eliminate the ().
- ◆ Strings are in double quotes ("..."). It is stored similar to C.
- ◆ No built-in limit on the size of a string.
- ◆ There is also a single quoted string.

Simple Rules to start with

- ◆ There is no need to declare variables.
- ◆ Scalar variable has a leading \$.
- ◆ Backslashed characters are similar to C.
"\\n" is the newline character.

Filehandler

- ◆ Standard file handlers: STDIN, STDOUT, STDERR, ARG.
- ◆ Any time we assign `<STDIN>` to a variable, it grabs a line of input (including the end-of-line character).
- ◆ `chop(s)` removes the last character from the string `s`.

Example

```
print "What is your name? ";  
$name = <STDIN>;  
chop ($name);  
print "Hello, $name!\n";
```

IF statement

```
if ( ) {  
    ...  
} elseif {  
    ...  
} else {  
    ...  
}
```

Operators

Numerical	String
==	eq
<	lt
>	gt
<=	le
>=	ge
!=	ne

Elsif Example

```
#!/usr/local/bin/perl
print "What is your name? ";
chop ($name=<STDIN>);
if ($name eq "Stephen") {
    print "Welcome, $name.\n";
} elsif ($name eq "Jessica") {
    print "How are you, $name.\n";
} else {
    print "What are you doing here?\n";
}
```

Arrays

- ◆ Array variable names begin with `@`, so they are different from scalar variable names.
- ◆ `$name` and `@name` are two different variables.
- ◆ `@name = ("Adam", "Bob", "Charlie");`

Associative Arrays

- ◆ We access the value of the array by providing a *key* instead of an *index* to it.
- ◆ Use % prefix instead of @.
- ◆ %student = ("name", "David", "height", 6.1, "degree", "Ph.D.");

String Matching

- ◆ To match a string with `$_`, use `/string/`. The result is true if the string appears anywhere in the variable.

```
print "What is your URL? ";  
chop ($_=<STDIN>);  
if (/seerakademi\.com/) { ## Yes, our  
    students  
} else {                ## No, outsider  
}
```

Regular Expressions

- ◆ Perl provides every regular expression feature found in standard Unix utility.

```
print "What is your URL? ";
chop ($url=<STDIN>);
if ($url =~ /seerakademi\.com$/) {
    ## Yes, our students
} else {
    ## No, outsider
}
#Matches "seerakademi.com" at the end of the
variable.
```

Examples

- ◆ \$name =~ /^John/ Search John in the beginning
- ◆ \$name =~ /^John\b/ \b is blank
- ◆ \$name =~ /^John/i Ignore case
- ◆ \$name =~ **s/^http/ftp/i** search and replace
- ◆ \$name =~ **tr/A-Z/a-z/** transliteration

Functions

◆ Function Definition

```
sub print_header {  
    print "Content-type:  
    text/html\n\n";  
}
```

◆ Function Call

```
&print_header(); or  
&print_header;
```

Loops

- A major reason we use computers is that they can do the same thing many times without getting tired or needing coffee. We get them to repeat actions using **iterative** or **looping constructs**.
- There are three main types of loop statement, which are all related but specialized for different uses: **for**, **foreach**, and **while statements**.

'while' loops

- **While statements** are the least complex of the three types, so we'll start there. They are similar to conditional statements, because they have a condition and a block. If the condition evaluates to true, the block is executed. Then, the condition is evaluated again, and **if it's still true**, the block is executed again, and so on.

```
while ($money < 1000) {  
    $pay = Get_Paycheck ();  
    $money = $money + $pay;  
}
```

This represents some user-defined subroutine, which presumably checks for more money and returns the value as **\$pay**.

```
print "Ok, we can pay the rent!\n";
```

'until' loops

- **Until statements** are a special type of **while** statement in which the block is executed **until the condition evaluates as true**. It's equivalent to "while NOT (condition)...".

```
until ($money < 1000) {  
    $money = Eat_Out ($money);  
}  
print "Time to eat noodles!\n";
```


'for' loops

- **For statements** are very useful, because they allow you to do something a preset number of times, or until a condition becomes true.
- Their structure is more complex than a while statement's:

```
for ($i = 1; $i <= 100; $i++) {  
    print "We've reached $i\n";  
}  
  
We've reached 1  
We've reached 2  
...  
We've reached 100
```

'foreach' loops

- **Foreach statements** allow us to perform the same operations on every element of an array or hash in series. They temporarily assign the value of each element to a scalar that we can work with.

```
foreach $grade (@grades) {  
    if ($grade < 60) {  
        print "Failing grade!: $grade\n";  
        $fail_tally ++;  
    }  
}  
  
print "We have $fail_tally failing  
students!\n";
```

Exiting the loop

- Sometimes you want to exit the loop via a method other than the standard one. We do this using the statements **next**, **redo**, and **last**.
- **next** means "skip over everything else in the block, increment the counter, and evaluate the conditional again."
- **redo** means "skip over everything else in the block and evaluate the conditional again, without incrementing the counter."
- **last** means "exit the block and never come back."

Exiting the loop

- Here's an example that uses both **next** and **last**:

```
foreach $student (@students) {  
    if ($student eq "END_REGISTERED") {  
        last;  
    }  
    elsif ($student eq "Silber") {  
        next;  
    }  
    else {  
        $grade = Check_Grade ($student);  
    }  
    print "$student: $grade\n";  
}
```

Reading from a File

```
open (LOG, 'access.log' );  
while (<LOG>) {  
    chop;  
    print "Accessed host: $_.\n";  
}
```

Writing to a File

```
open (MYFILE, '> MyFile.txt') ;  
print MYFILE "Welcome.\n";  
...  
close (MYFILE) ;
```